

Traditional On-Premises vs Hybrid vs Full Cloud-Native

The Complete Architecture Guide — Tools, Flows, Security & Migration

What Really Happens When a User Opens a URL — Full Internal Request Lifecycle

On-Premises → Hybrid → Full Azure Cloud-Native, With Real Diagrams for Each

Migrating On-Premises to Azure — Strategies, Steps, End-to-End Flow

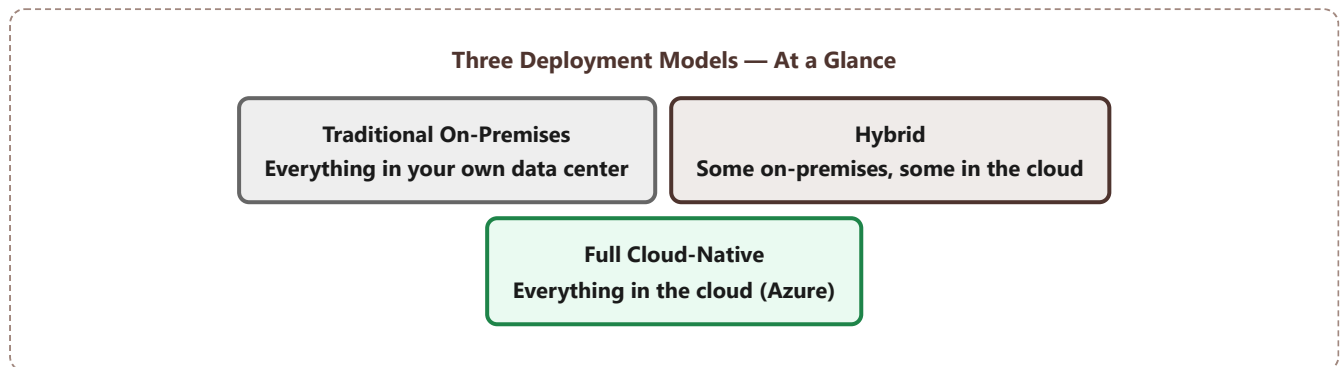
A practical, simple-language, end-to-end reference for interview preparation and real-world implementation

Table of Contents

1. Introduction — Traditional, Hybrid and Full Cloud at a Glance
2. Traditional On-Premises Architecture — Full Deep Dive
3. On-Premises Tools & Technology Stack — What Each Tool Is and Why It's Used
4. On-Premises Request Flow — Full Internal Working, Step by Step
5. On-Premises Deployment Flow (Git → Jenkins → Docker → Servers)
6. On-Premises Security — Full Deep Dive
7. On-Premises Monitoring (Splunk → Grafana → Zipkin)
8. What Is Hybrid Cloud and Why Organizations Use It
9. Hybrid Architecture — Full Deep Dive
10. Hybrid Request Flow — Full Internal Working
11. Hybrid Security & Networking (VPN, ExpressRoute)
12. Hybrid Cloud Real Scenarios & Migration Considerations
13. Full Cloud-Native Architecture — Full Deep Dive
14. Where React, Backend and Database Actually Get Deployed in Full Cloud
15. Full Cloud Request Flow — Complete Internal Working (URL to Response)
16. Full Cloud Security (Key Vault, Entra ID)
17. Full Cloud Monitoring & Observability
18. Full Cloud CI/CD Flow (Git → Azure DevOps → Docker → ACR → AKS)
19. Traditional vs Hybrid vs Full Cloud — Full Comparison (Cost, Performance, Maintenance)
20. Migrating On-Premises to Azure — Migration Strategies (The 6 R's)
21. Migration Steps End to End
22. Migration Real Scenarios, Issues & Solutions
23. What Happens When You Open a URL — Basic Introduction With a Real Example
24. What Happens When You Open a URL — Full Advanced Internal Working
25. Interview Questions & Real-Time Scenario-Based Q&A
26. Final Summary & Cheat Sheet

1. Introduction — Traditional, Hybrid and Full Cloud at a Glance

1.1 The Three Architecture Models This Guide Covers



1.2 Why Understanding All Three Matters

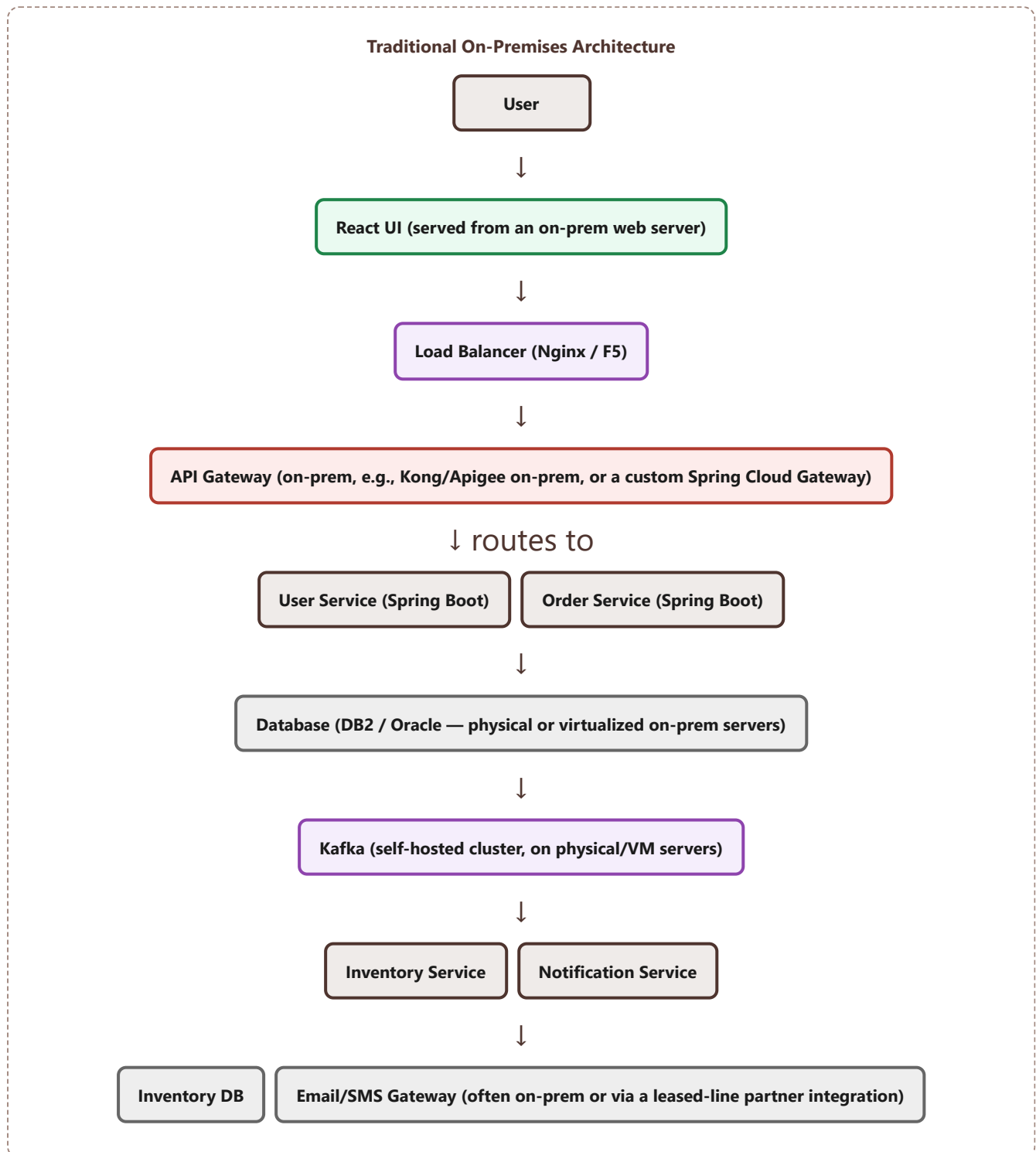
Very few real organizations jump straight from "nothing" to "100% cloud-native" — most walk through all three stages over years, and many large enterprises (banks, insurance, government) still run significant Hybrid architectures today for regulatory, legacy, or cost reasons. Understanding all three — and how to move between them — is essential real-world architecture knowledge, not just a historical curiosity.

1.3 How This Guide Is Structured

- **Sections 2-7:** Traditional On-Premises — full architecture, tools, request flow, deployment, security, monitoring.
- **Sections 8-12:** Hybrid Cloud — what it is, architecture, request flow, security/networking, real scenarios.
- **Sections 13-18:** Full Cloud-Native — full architecture (matching your real Azure diagram), where each piece deploys, request flow, security, monitoring, CI/CD.
- **Section 19:** Full comparison across cost, performance, and maintenance.
- **Sections 20-22:** Migrating on-premises to Azure — strategies, steps, real scenarios.
- **Sections 23-24:** A dedicated deep dive on exactly what happens, technically, when a user opens a URL — from basic to fully advanced.
- **Sections 25-26:** Interview questions, real scenarios, and a final cheat sheet.

2. Traditional On-Premises Architecture — Full Deep Dive

2.1 The Real Traditional Architecture Diagram

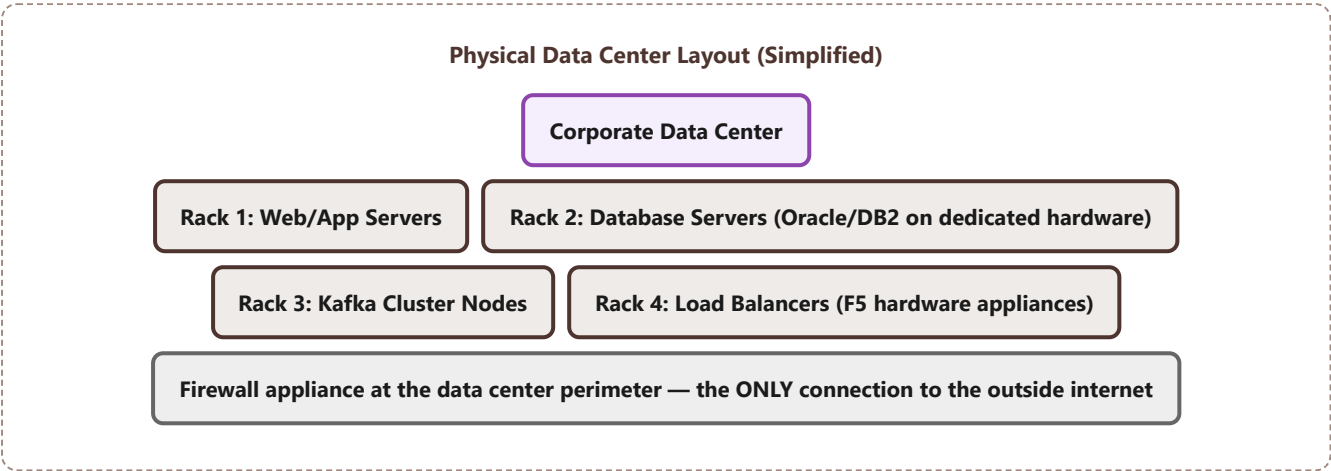


2.2 What "On-Premises" Really Means Physically

Every single box in the diagram above is a PHYSICAL (or virtualized-on-physical) server sitting in the organization's own data center (or a rented colocation facility) — owned or leased hardware, racked, cabled, powered, and cooled by the

organization's own infrastructure team, with no cloud provider involved anywhere in this picture.

2.3 Real Physical/Logical Layout



2.4 Real Organizational Ownership

Component	Who Owns/Manages It
Physical servers, racks, power, cooling	Internal Infrastructure/Data Center Operations team
Network switches, F5 load balancers, firewalls	Internal Network Engineering team
Database (Oracle/DB2) administration	Internal DBA team
Application code (Spring Boot services, React)	Development teams
Deployment/release process	Internal Release Engineering / Ops team, often manual or semi-automated

Real-world note: In a traditional model, EVERY layer requires a dedicated internal team with deep hardware/OS-level expertise — this is precisely the operational burden (Section 1 of the AWS/Azure guide) that cloud computing was created to remove.

3. On-Premises Tools & Technology Stack — What Each Tool Is and Why It's Used

3.1 Full Tool Reference Table

Tool/Component	What It Is	Why It's Used Here	Real Alternative
Nginx / F5	Nginx: software load balancer/reverse proxy. F5: a dedicated hardware load balancing appliance	Distributes incoming traffic across multiple app server instances, provides SSL termination	HAProxy, Citrix ADC
API Gateway (on-prem)	A centralized entry point for API traffic — often Kong, Apigee (self-hosted), or a custom Spring Cloud Gateway	Centralizes auth, routing, rate limiting for all backend APIs	WSO2 API Manager, Mulesoft
Spring Boot Services	The actual Java business logic microservices/monolith	Implements User Service, Order Service business logic	—
DB2 / Oracle	Enterprise-grade commercial relational databases	Strong ACID transactions, mature enterprise tooling, long-standing organizational investment/licensing	PostgreSQL, SQL Server
Kafka (self-hosted)	A distributed event streaming platform, run on your own servers/VMs	Asynchronous decoupling between core services and downstream reactive services (Microservices guide, Section 24, 36)	RabbitMQ, IBM MQ
Jenkins	A self-hosted, open-source CI/CD automation server	Automates build/test/deploy pipelines, runs entirely on infrastructure you control	GitLab CI (self-hosted), Bamboo
Docker	Containerization platform (Docker & Kubernetes guide, full detail)	Packages the app consistently even in an on-prem deployment target	Podman
Splunk	Enterprise log aggregation and search platform (Observability guide, Section 13)	Centralized log search across all on-prem servers	Self-hosted ELK Stack
Grafana	Metrics dashboarding platform (Observability guide, Section 23)	Visualizes metrics collected from on-prem servers/applications	Kibana (for log-based visualization)
Zipkin	Distributed tracing tool (Observability guide, Section 26)	Traces a request across the on-prem microservices	Jaeger

3.2 Why These Specific Tools in a Traditional Setup

Notice every single tool above is either open-source (self-hosted, no cloud dependency) or a commercial on-prem license (Oracle, DB2, F5) — this reflects the traditional model's core constraint: nothing can depend on any external cloud service, since the entire architecture must run fully self-contained within the organization's own data center.

4. On-Premises Request Flow — Full Internal Working, Step by

Step

4.1 The Complete Journey of One Real Request

On-Premises Request — Full Internal Flow

1. User's browser sends HTTPS request to the company's public-facing domain

↓ DNS resolves to the data center's public IP (Web/Network Basics guide, Section 2.2)

2. Corporate Firewall inspects and allows the traffic through



3. F5/Nginx Load Balancer terminates TLS, picks a healthy web server



4. React UI's static files are served (or, for API calls, forwarded onward)



5. On-prem API Gateway authenticates the request, routes based on path



6a. User Service (Spring Boot) handles /users/*

6b. Order Service (Spring Boot) handles /orders/*



7. Service queries Oracle/DB2 over the internal network (private VLAN)

↓ if the action triggers a downstream event

8. Service publishes a message to the self-hosted Kafka cluster



9a. Inventory Service consumes and updates Inventory DB

9b. Notification Service consumes and triggers Email/SMS



10. Response flows back through the same path — Service → API Gateway → Load Balancer → back to the user's browser

4.2 What Happens INSIDE the Spring Boot Service at Step 6-7 (Recap)

Exactly the same internal Spring Boot request lifecycle already covered in complete depth in the Web/Network/Startup/Redis guide (Section 21) — DispatcherServlet, HandlerMapping, Controller, @Transactional Service layer, Repository/JPA/Hibernate hitting the database — this lifecycle is IDENTICAL regardless of whether the service happens to be running on-premises or in the cloud; only the surrounding infrastructure differs.

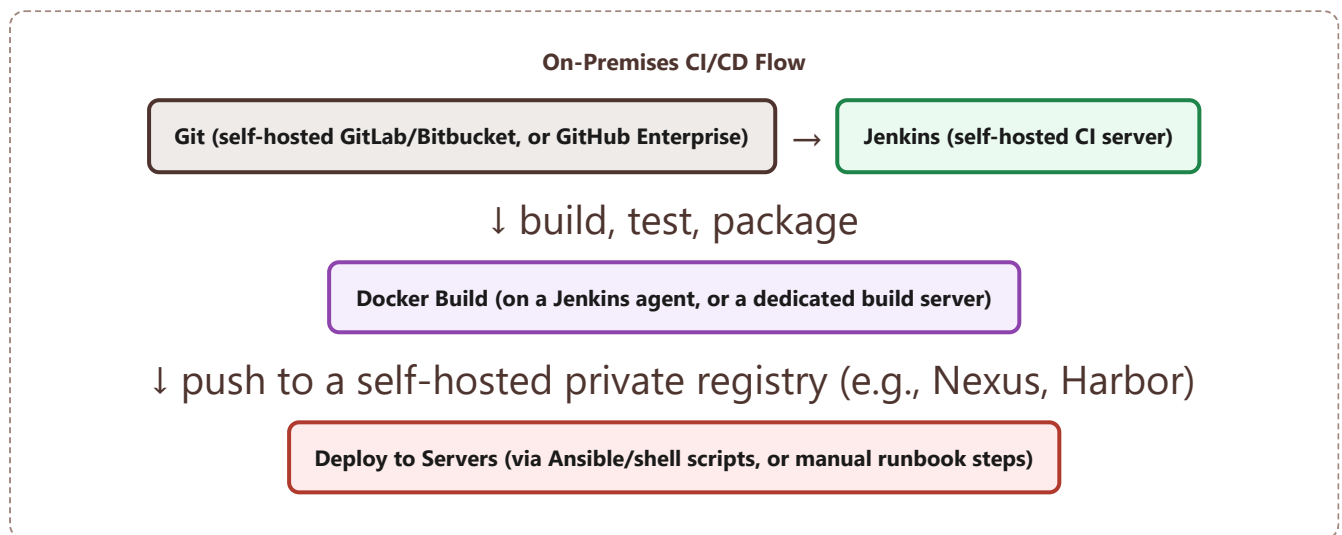
4.3 Real Latency Characteristics On-Premises

Hop	Typical Latency	Why
User to Data Center (internet)	20-150ms, depending on user's geographic distance	No CDN/edge caching typically available in a pure on-prem setup
Load Balancer to App Server	< 1ms	Same physical data center, low-latency internal network
App Server to Database	< 1ms to a few ms	Same data center, dedicated high-speed network links

Real scenario: "A company with a single data center in New York serving customers in Singapore experienced 200ms+ latency purely from the physical distance the request had to travel — with no CDN or regional presence, every single request paid this full round-trip cost, a real limitation that pushed the organization toward at least a Hybrid model (Section 8) for global reach."

5. On-Premises Deployment Flow (Git → Jenkins → Docker → Servers)

5.1 The Real Deployment Pipeline



5.2 What Actually Happens at Each Stage

1. **Git:** developer pushes code to a self-hosted Git server (no cloud-hosted GitHub/Azure Repos dependency in a fully on-prem shop).
2. **Jenkins:** a webhook or polling trigger kicks off a pipeline job — compiles, runs unit tests, and often SonarQube (self-hosted) quality gates.
3. **Docker Build:** Jenkins agent builds the Docker image on its own hardware, tags it, pushes to a self-hosted private registry.
4. **Deploy to Servers:** a deployment script (often Ansible, or in less mature setups, a manual SSH-and-run-script process) pulls the new image and restarts the service on each target server.

5.3 Real Limitations of This Pipeline (Motivating Later Cloud Adoption)

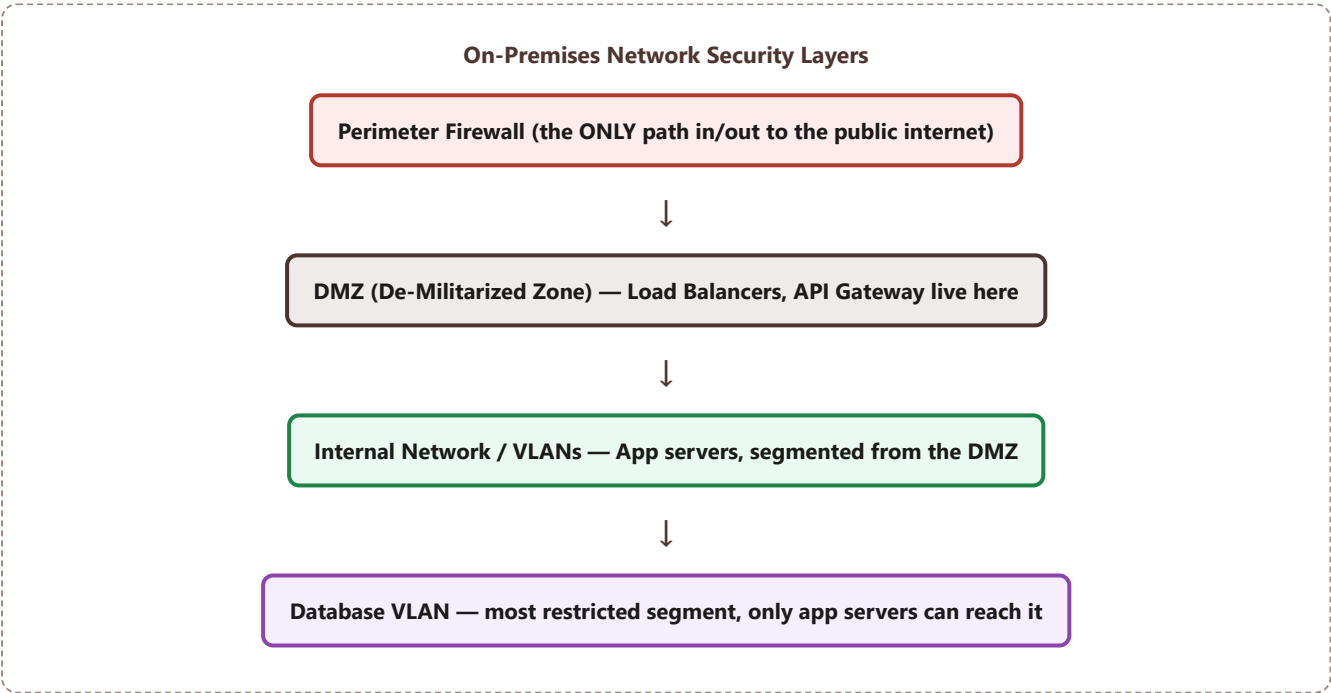
Limitation	Real Impact
Jenkins itself needs its own dedicated server(s) to maintain, patch, and scale	Additional infrastructure burden on top of the application infrastructure itself
No elastic build capacity	A surge of concurrent builds queues up, waiting for available Jenkins agents
Manual/scripted deployment to a FIXED set of servers	No true auto-scaling of the deployment target itself — capacity is whatever's physically racked
Rollback often means manually redeploying a previous version	Slower, more error-prone recovery compared to Kubernetes' instant rollout undo (Docker & Kubernetes guide, Section 21.4)

6. On-Premises Security — Full Deep Dive

6.1 The Physical Security Perimeter

On-premises security starts with genuinely PHYSICAL controls — badge-access data center doors, security cameras, biometric locks on server racks — a layer that simply doesn't exist in the same form once you move to the cloud (where physical security becomes the cloud provider's responsibility).

6.2 Network Security Layers



6.3 Identity & Access Management On-Premises

Concern	Real On-Prem Tool
Employee/system identity	Microsoft Active Directory (the on-prem predecessor to Azure Entra ID)
Application secrets (DB passwords, API keys)	Often a self-hosted HashiCorp Vault, or (in less mature setups) config files with restricted file permissions
Encryption at rest	Database-native Transparent Data Encryption (TDE), disk-level encryption
VPN for remote developer/admin access	Self-hosted VPN concentrator/appliance

6.4 Real Security Trade-Offs of the On-Premises Model

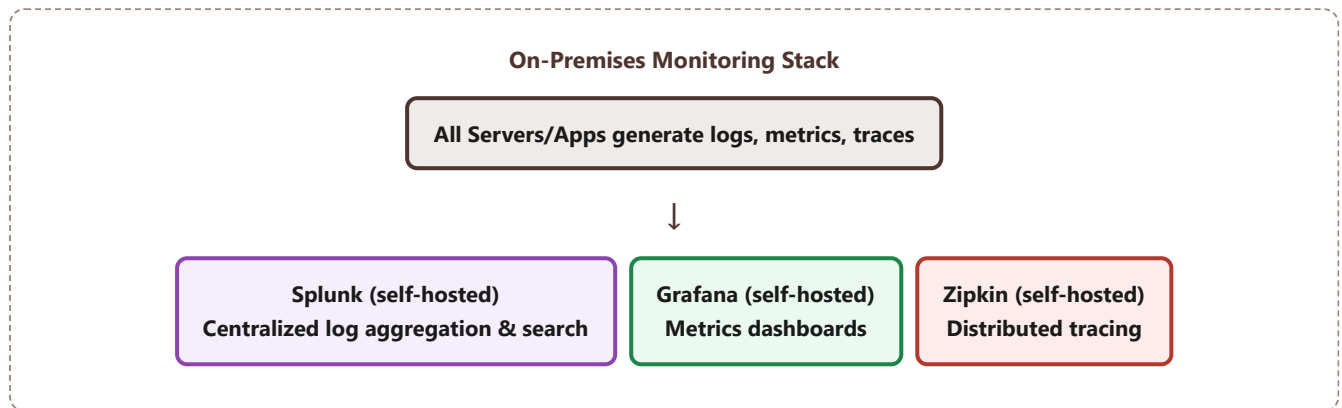
Advantage	Disadvantage
-----------	--------------

Complete physical control over hardware — no third-party ever has physical access	Your organization bears 100% of the responsibility for patching, physical security, and disaster recovery
Data never leaves your own network boundary (a common regulatory requirement in some industries)	No access to cloud-native, constantly-updated security services (WAF, DDoS protection at cloud scale) without building/buying your own equivalent
Simpler compliance story for certain strict data-residency regulations	Security team must have deep expertise across networking, OS hardening, and application security all at once

Real scenario: "A regulated financial institution kept its core ledger database fully on-premises specifically because of a strict data-residency requirement mandating that certain data never leave a specific physical jurisdiction — this exact regulatory constraint is a common real reason organizations retain SOME on-premises footprint even today, motivating the Hybrid model (Section 8) rather than full cloud migration."

7. On-Premises Monitoring (Splunk → Grafana → Zipkin)

7.1 The Real On-Premises Observability Flow



7.2 Why Three Separate Self-Hosted Tools

Exactly the Three Pillars of Observability pattern covered in complete depth in the Observability & Monitoring guide (Section 7) — Splunk for logs, Grafana (typically backed by a self-hosted Prometheus) for metrics, Zipkin for traces — the ONLY difference in a traditional on-premises setup is that every single one of these tools must be installed, patched, scaled, and maintained on your own servers, rather than consumed as a managed cloud service.

7.3 Real Operational Burden of Self-Hosted Observability

Task	Who Does It On-Premises	Who Does It in Full Cloud (Section 17)
Patching Splunk/Grafana/Zipkin servers	Your own infrastructure team	Largely eliminated — Azure Monitor/App Insights are fully managed
Scaling the observability stack for growing data volume	Manually provisioning more storage/compute for these tools	Auto-scales as part of the managed service
High availability of the monitoring stack itself	You must design and maintain redundancy for Splunk/Grafana/Zipkin themselves	Built into the managed service's SLA

Real irony worth knowing: Your monitoring/observability stack ITSELF can become a single point of failure requiring its own monitoring and redundancy planning in a self-hosted on-premises setup — a genuine operational burden layer that's easy to underestimate until you've maintained one.

8. What Is Hybrid Cloud and Why Organizations Use It

8.1 The Core Definition

Hybrid Cloud means running SOME workloads on-premises and OTHERS in a public cloud (Azure/AWS), with the two environments connected and working together — not a "migration in progress" necessarily, but often a deliberate, long-term architectural choice.

8.2 Real Reasons Organizations Choose (or Stay) Hybrid

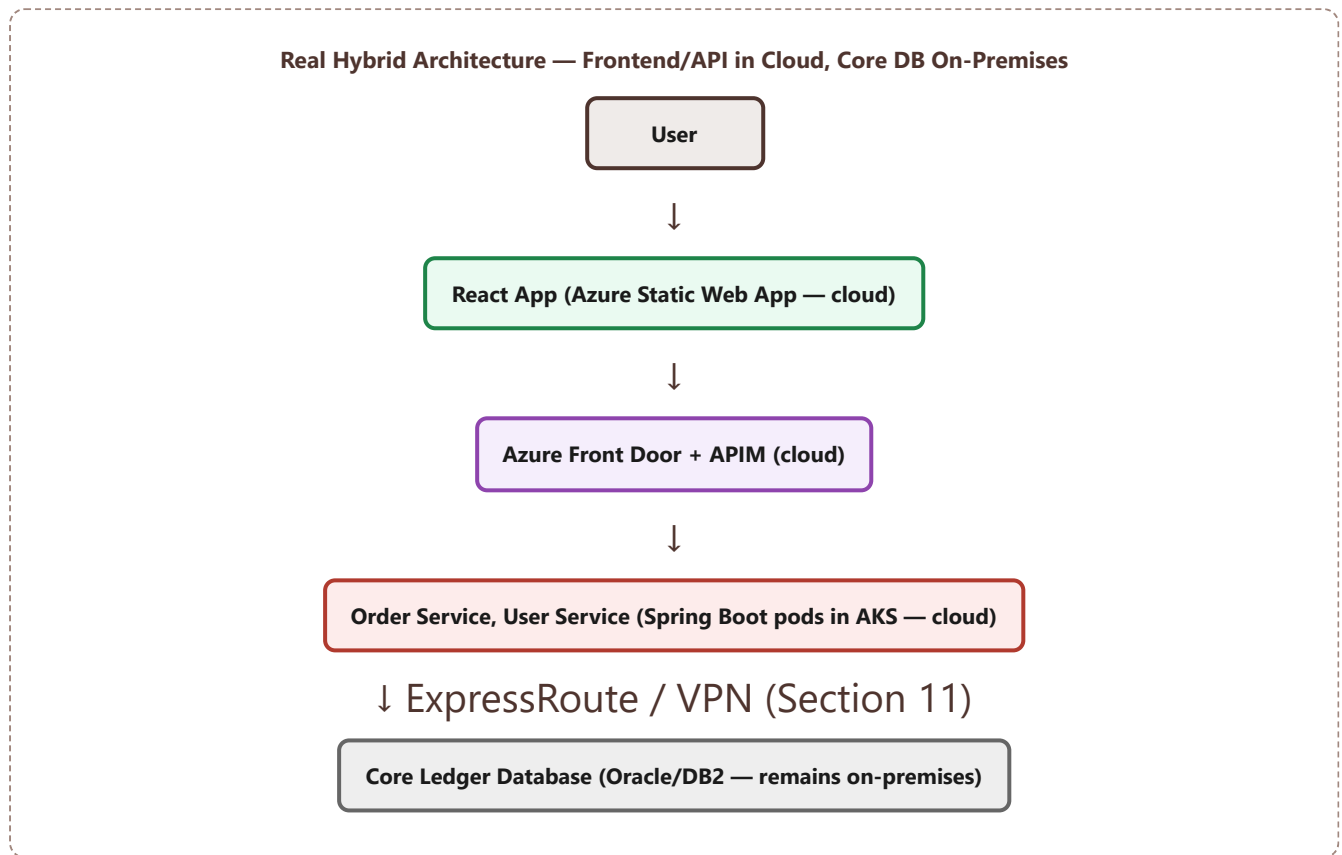
Reason	Real Example
Regulatory/data-residency requirements	Core ledger/customer PII data must legally remain on-premises in a specific jurisdiction
Legacy system dependencies	A mainframe or DB2 system too costly/risky to migrate, but new services should be cloud-native
Gradual migration strategy	Moving one service at a time to the cloud (Strangler Fig pattern, Microservices guide Section 9) while the rest stays on-prem
Burst capacity ("cloud bursting")	Core capacity stays on-prem (already paid for), but traffic SPIKES burst into the cloud temporarily
Existing capital investment	Recently purchased expensive on-prem hardware not yet fully depreciated

8.3 Hybrid Cloud vs "Cloud Migration in Progress" — An Important Distinction

Sometimes Hybrid is a permanent, deliberate end-state (e.g., regulatory data residency), and sometimes it's a temporary transitional phase during an active migration (Sections 20-22) — real architecture decisions must be clear about WHICH of these two scenarios actually applies, since the two call for very different levels of investment in the hybrid connectivity itself.

9. Hybrid Architecture — Full Deep Dive

9.1 A Real Hybrid Architecture Diagram



9.2 Common Real Hybrid Split Patterns

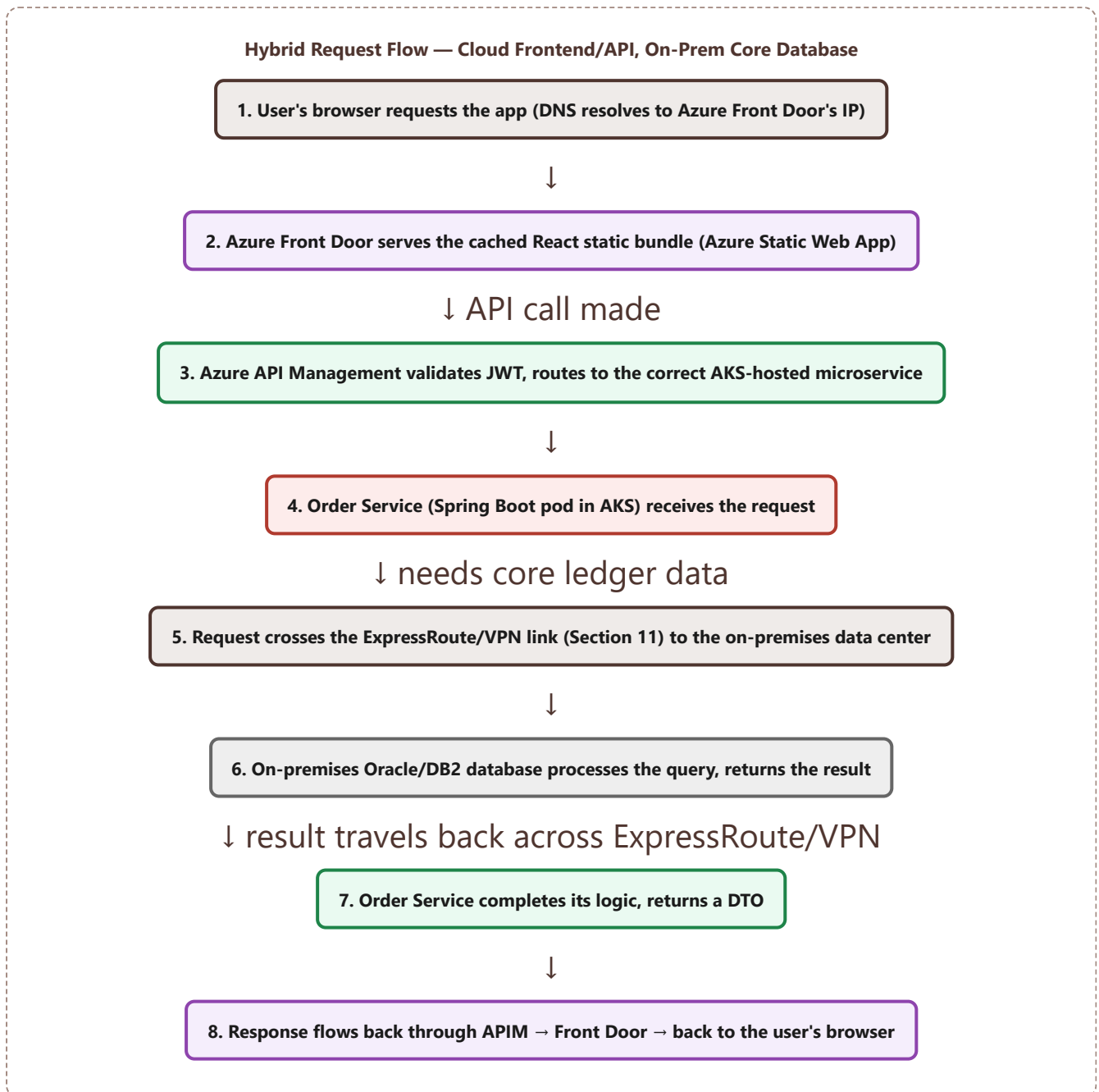
Pattern	What Stays On-Prem	What Moves to Cloud	Real Reason
Frontend-Cloud, Data-On-Prem	Core database, legacy mainframe integrations	React frontend, API Gateway, stateless microservices	Fastest way to modernize user experience without touching risky legacy data systems yet
Dev/Test-Cloud, Prod-On-Prem	Production workloads	Development and testing environments	Get elastic, low-cost dev/test capacity while production stability requirements are met on proven on-prem infrastructure
Burst-to-Cloud	Baseline/steady-state capacity	Overflow capacity during traffic spikes	Avoid over-provisioning on-prem hardware for rare peak events (Black Friday-style)
DR (Disaster Recovery)-in-Cloud	Primary production	A cloud-hosted disaster recovery replica	Cheaper and faster than building/maintaining a second physical data center

9.3 Real Data Synchronization Challenges

Whenever data or services span both environments, keeping them consistent becomes a real engineering challenge — network latency between on-prem and cloud (even over a dedicated ExpressRoute link) is inherently higher than within a single environment, and any service calling across that boundary must apply the full resilience pattern set (timeout, retry, circuit breaker — [Microservices guide Section 40](#)) even more rigorously than for purely intra-cloud calls.

10. Hybrid Request Flow — Full Internal Working

10.1 The Complete Real Hybrid Request Journey



10.2 The Real Latency Cost of Crossing the Hybrid Boundary

Connection Type	Typical Added Latency (Round Trip)
Azure ExpressRoute (dedicated private circuit)	Low, predictable — often single-digit to low double-digit milliseconds, depending on physical distance
Site-to-Site VPN over the public internet	Higher and less predictable — subject to general internet congestion/routing variability

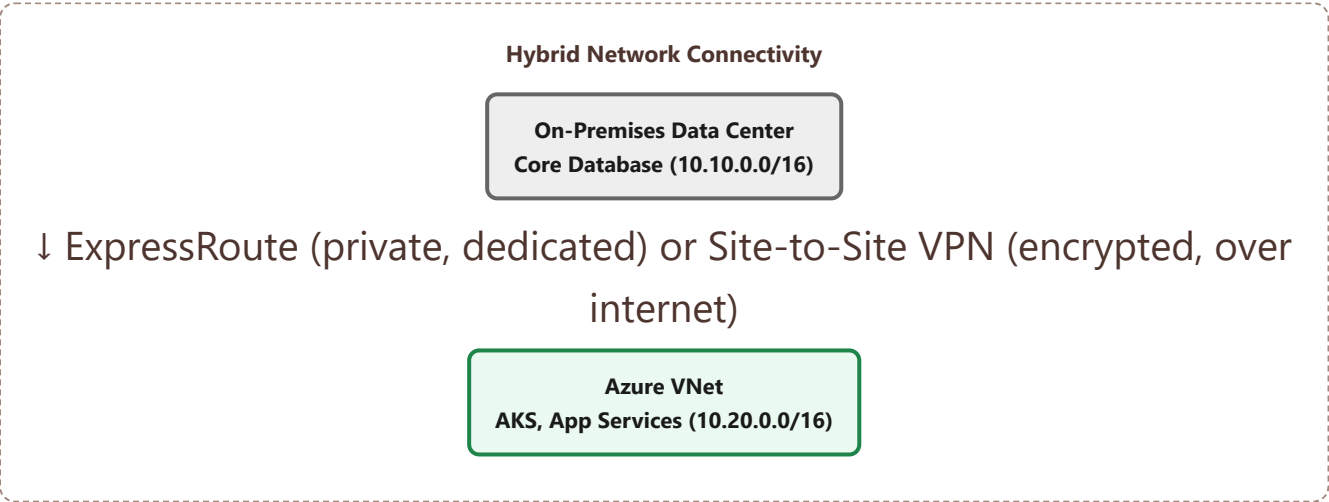
Real scenario: "Every single Order Service request that needs the on-premises ledger database pays a real, measurable ExpressRoute round-trip cost — our team specifically cached frequently-accessed, rarely-changing reference data (like product categories) in Redis within the AKS cluster (Section 30 pattern, AWS/Azure guide) precisely to avoid crossing this hybrid boundary for data that doesn't need to be fetched fresh every single time."

11. Hybrid Security & Networking (VPN, ExpressRoute)

11.1 The Two Real Connectivity Options

Option	How It Works	Real Trade-Off
Site-to-Site VPN	Encrypted tunnel over the PUBLIC internet connecting on-prem network to Azure VNet	Cheaper, faster to set up; subject to public internet variability/congestion
Azure ExpressRoute	A dedicated, private physical network connection from on-prem directly into Azure, NEVER touching the public internet	More expensive, requires a telecom provider partnership; much more predictable latency and higher security/compliance posture

11.2 Real Hybrid Network Diagram



11.3 Identity Federation Across the Hybrid Boundary

Real hybrid organizations typically federate their on-premises Active Directory with Azure Entra ID (Entra ID Connect / Azure AD Connect) — one single identity works seamlessly across BOTH environments, rather than maintaining two entirely separate, disconnected identity systems.

11.4 Real Issues & Solutions

Issue	Cause	Real Solution
Intermittent connectivity failures between cloud and on-prem	Site-to-Site VPN relying on unpredictable public internet routing	Upgrade to ExpressRoute for critical, latency-sensitive, or high-volume traffic paths
IP address range conflicts preventing connectivity	On-prem and Azure VNet CIDR ranges overlap	Plan non-overlapping IP ranges across both environments before establishing connectivity (AWS/Azure guide, Section 8.7)

Users experiencing inconsistent login behavior	Identity not properly federated between AD and Entra ID	Implement Entra ID Connect for seamless single-identity authentication across both environments
--	---	---

12. Hybrid Cloud Real Scenarios & Migration Considerations

12.1 Real Scenario — Phased Migration Using Hybrid as a Stepping Stone

"Our organization moved the React frontend and stateless API layer to Azure FIRST (low risk, immediate user experience/performance benefit via Front Door's global CDN), while the core ledger database stayed on-premises for another 18 months while the DBA team carefully planned and executed its own migration (Sections 20-22) — this phased Hybrid approach let us deliver real, immediate value to users without waiting for the hardest, highest-risk part of the migration (the core database) to be ready."

12.2 Real Scenario — Permanent Regulatory Hybrid

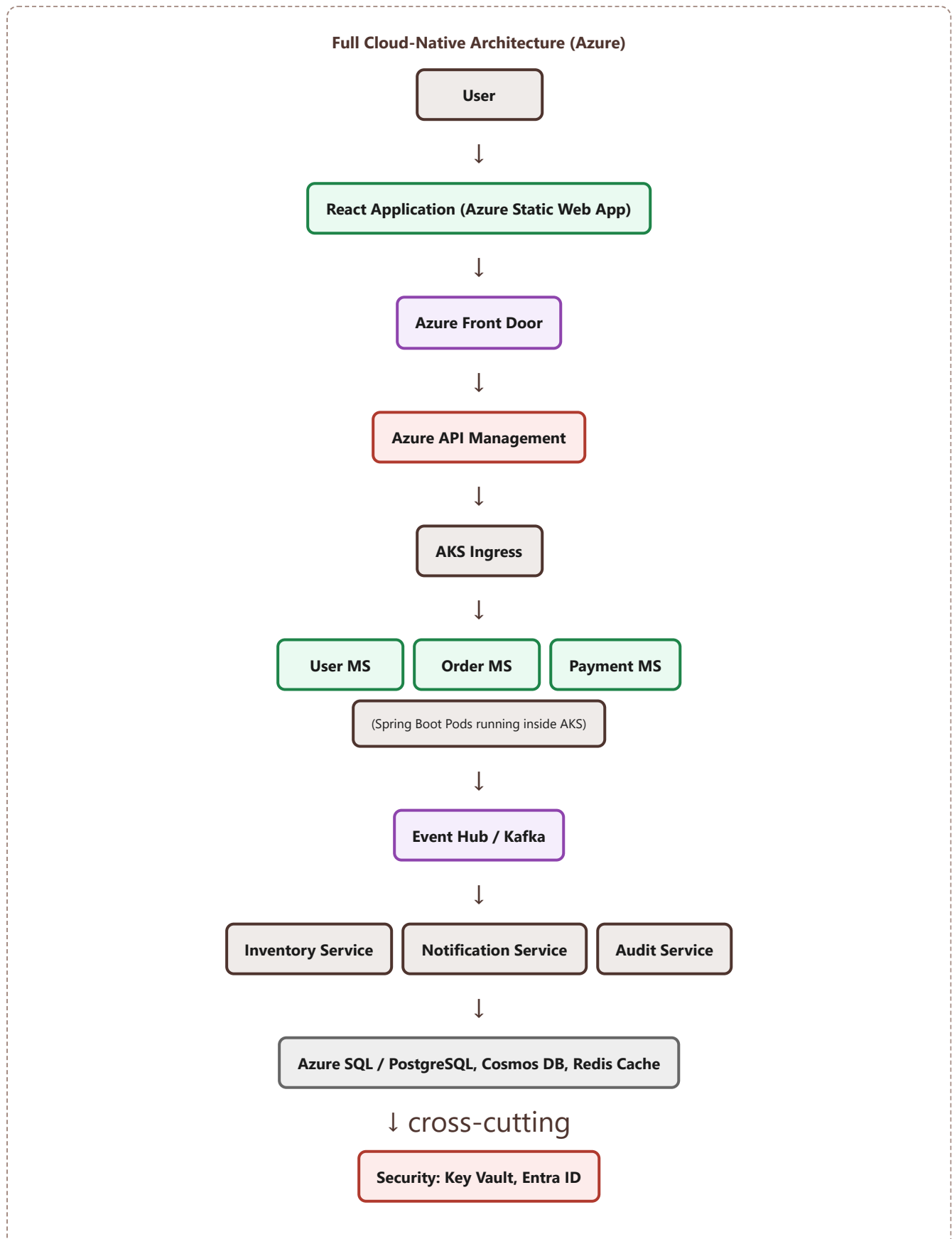
"A healthcare/financial client's core PII database remains PERMANENTLY on-premises due to strict data-residency regulation in their jurisdiction — this isn't a migration-in-progress, it's the organization's deliberate, permanent target architecture, with the cloud used for everything that ISN'T subject to that specific regulatory constraint."

12.3 Key Decision Factors for Choosing (or Staying) Hybrid

- Regulatory/compliance requirements mandating specific data stay within a jurisdiction or specific physical control.
- Existing capital investment in on-prem hardware not yet depreciated.
- Legacy system migration risk/cost genuinely outweighing the benefit of moving it, at least for now.
- Organizational readiness/skills — a gradual hybrid transition lets teams build cloud expertise incrementally rather than all at once.

13. Full Cloud-Native Architecture — Full Deep Dive

13.1 The Real Full Cloud-Native Architecture Diagram



Monitoring: Azure Monitor, Application Insights, Grafana, Splunk

CI/CD: Git → Azure DevOps → Docker Build → ACR → AKS Deployment

13.2 What's Genuinely Different From the On-Premises/Hybrid Models

Aspect	On-Premises	Full Cloud-Native
Infrastructure ownership	You own/manage every physical layer	Azure manages all physical infrastructure; you manage configuration/code
Scaling	Manual capacity planning, physical hardware limits	Auto Scaling at every layer (AKS HPA, Cluster Autoscaler, VMSS)
Global reach	Limited to your data center's physical location(s)	Front Door's global edge network reaches users worldwide instantly
Disaster recovery	Requires building/maintaining a second physical data center	Multi-region deployment achievable through configuration, not new hardware
Tooling	Self-hosted Jenkins, Splunk, Zipkin — all self-maintained	Managed equivalents (Azure Pipelines, Azure Monitor, App Insights) — provider-maintained

14. Where React, Backend and Database Actually Get Deployed in Full Cloud

14.1 React Frontend — Deployment Location

React's built static bundle (HTML/CSS/JS, produced once via `npm run build`) is deployed to **Azure Static Web Apps** — a service specifically designed to host static frontend content, with built-in global CDN distribution, automatic HTTPS, and direct GitHub/Azure DevOps integration for automatic deployment on every push.

```
# Real Azure Static Web App deployment via Azure DevOps/GitHub Actions
- task: AzureStaticWebApp@0
  inputs:
    app_location: "/customer-portal"
    output_location: "build"
    azure_static_web_apps_api_token: $(deployment_token)
```

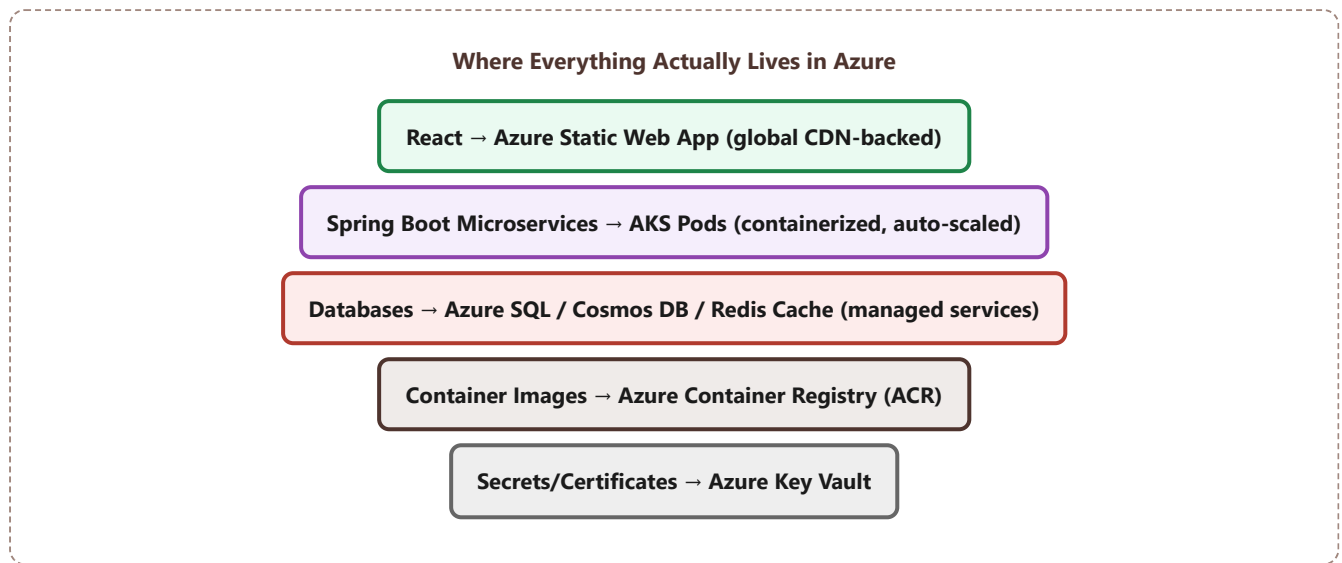
14.2 Spring Boot Backend — Deployment Location

Every Spring Boot microservice (User MS, Order MS, Payment MS) is containerized (Docker & Kubernetes guide, full detail) and deployed as its own Kubernetes Deployment/Pods running inside **AKS (Azure Kubernetes Service)** — each with its own Service, HPA, and resource limits, exactly per the Microservices guide's real organizational pattern.

14.3 Database — Deployment Location

Data Type	Real Azure Service Used	Why
Core relational data (Orders, Payments, Users)	Azure SQL Database / Azure Database for PostgreSQL	Strong ACID transactions, mature relational tooling (SQL guide, full detail)
Flexible-schema, high-throughput data (Inventory, catalogs)	Cosmos DB	Horizontal scale, flexible schema (AWS/Azure guide, Section 15.3)
Fast, shared caching layer	Redis Cache (Azure Cache for Redis)	Sub-millisecond access for frequently-read, rarely-changed data (Web/Network/Redis guide, full detail)

14.4 The Complete Deployment Map



15. Full Cloud Request Flow — Complete Internal Working

(URL to Response)

15.1 The Complete Real Request Journey, Every Internal Detail

Full Cloud-Native Request — Complete Internal Flow

1. User types the URL / clicks a link — browser performs DNS lookup

↓ resolves to Azure Front Door's anycast IP (nearest edge location, globally)

2. TLS handshake with Front Door's edge node (Web/Network guide, Section 3.3)



3. Front Door serves the **CACHED** React static bundle directly from the edge — near-instant, no origin round trip needed

↓ React app loaded, makes an API call (e.g., GET /api/v1/orders/1001)

4. This API request is **NOT** cacheable — Front Door forwards it to Azure API Management



5. APIM validates the JWT (issued by Entra ID) — invalid/expired token rejected here immediately

↓ valid token, rate limit checked and passed

6. APIM applies routing policy, forwards to AKS Ingress



7. AKS Ingress Controller (NGINX Ingress) routes based on path to the correct Kubernetes Service



8. Kubernetes Service load-balances to one healthy Order MS Pod

↓ INSIDE the pod — full Spring Boot internal request lifecycle (recap, Web/Network/Startup guide Section 21)

9. Servlet Filter Chain (correlation ID, Spring Security) → DispatcherServlet → Controller → Service (@Transactional) → Repository

↓ checks Redis Cache first (cache-aside pattern)

10a. Cache HIT: return immediately, no DB call — OR — 10b. Cache MISS: query Azure SQL/Cosmos DB, then populate the cache

↓ if the action triggers a downstream event

11. Order MS publishes an event to Event Hub/Kafka

↓ consumed independently, asynchronously, NOT blocking this request's response

Inventory Service

Notification Service

Audit Service

↓ meanwhile, Order MS's response is already flowing back

12. DTO serialized to JSON, response flows back: Pod → Ingress → APIM → Front Door → User's browser

↓ throughout this ENTIRE flow

13. Application Insights/Azure Monitor captured a distributed trace span at EVERY single hop above, correlated by one trace ID

15.2 Real Timing Breakdown (Typical Numbers)

Hop	Typical Latency
DNS resolution (often cached by the browser/OS)	0-20ms
TLS handshake with Front Door edge	10-50ms (faster with TLS session resumption)
Front Door → APIM → AKS Ingress	5-20ms (all within Azure's own backbone network)
Spring Boot processing (cache hit)	1-10ms
Spring Boot processing (cache miss, DB query)	10-50ms depending on query complexity/indexing
Total typical round trip (warm cache)	Often under 100ms end to end

16. Full Cloud Security (Key Vault, Entra ID)

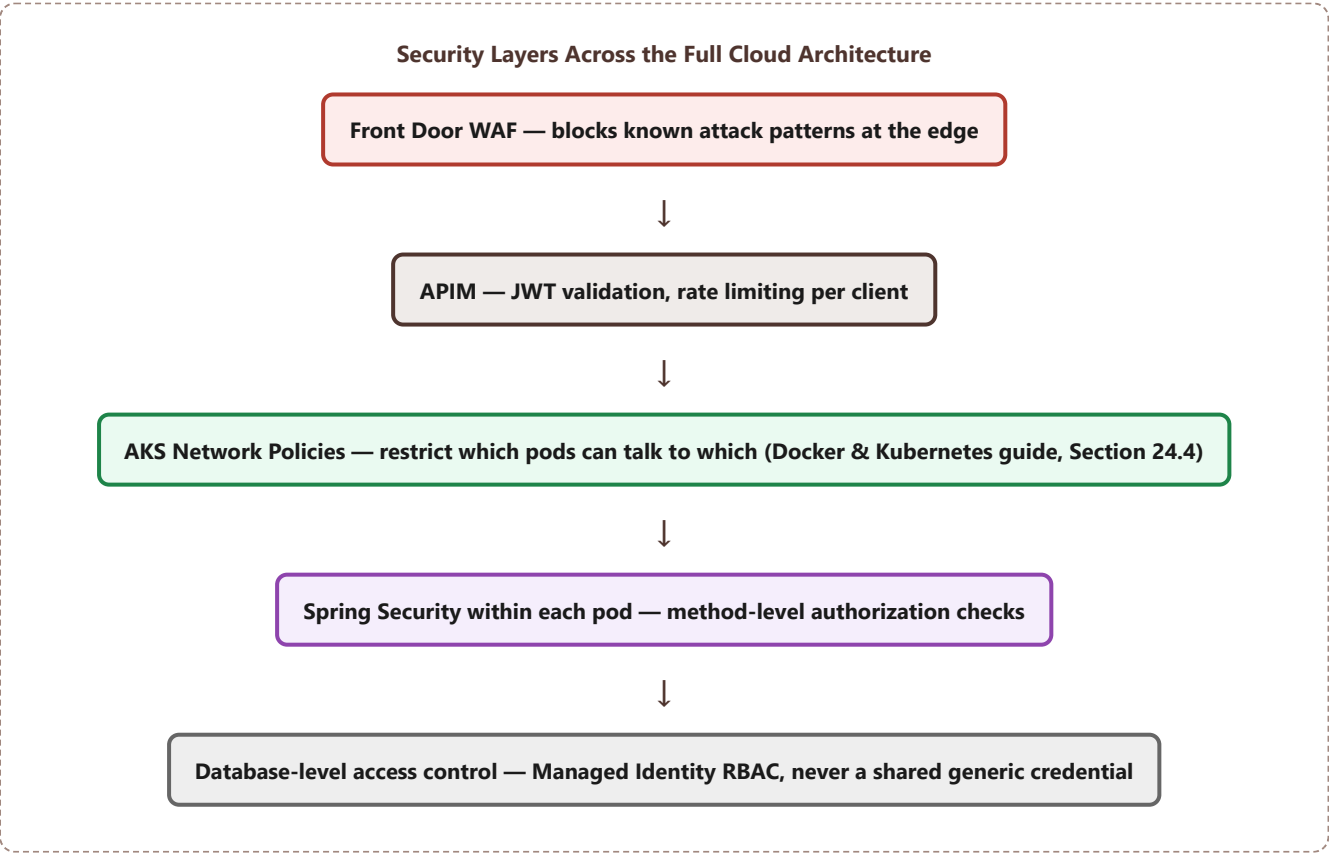
16.1 Key Vault — Full Recap in This Architecture's Context

Every single secret used anywhere in this architecture — database connection strings, JWT signing keys, third-party API keys, TLS certificates — is stored in Azure Key Vault, never hardcoded, accessed via Managed Identity (AWS/Azure guide, Section 9.5) with zero credentials ever appearing in code, config files, or CI/CD variables in plaintext.

16.2 Entra ID — Identity for Both Humans and Applications

Identity Type	Real Use in This Architecture
Human users (customers)	Authenticate via Entra ID B2C (or a custom identity flow), receive a JWT used for every API call
Human users (developers/admins)	Authenticate to the Azure Portal, AKS, and Azure DevOps via their corporate Entra ID account
Applications (Spring Boot services)	Each AKS pod uses a Managed Identity, granted least-privilege RBAC roles (AWS/Azure guide, Section 9.3) to access Key Vault, Cosmos DB, etc.

16.3 Full Defense-in-Depth Security Layering in This Architecture



17. Full Cloud Monitoring & Observability

17.1 The Real Managed Observability Stack

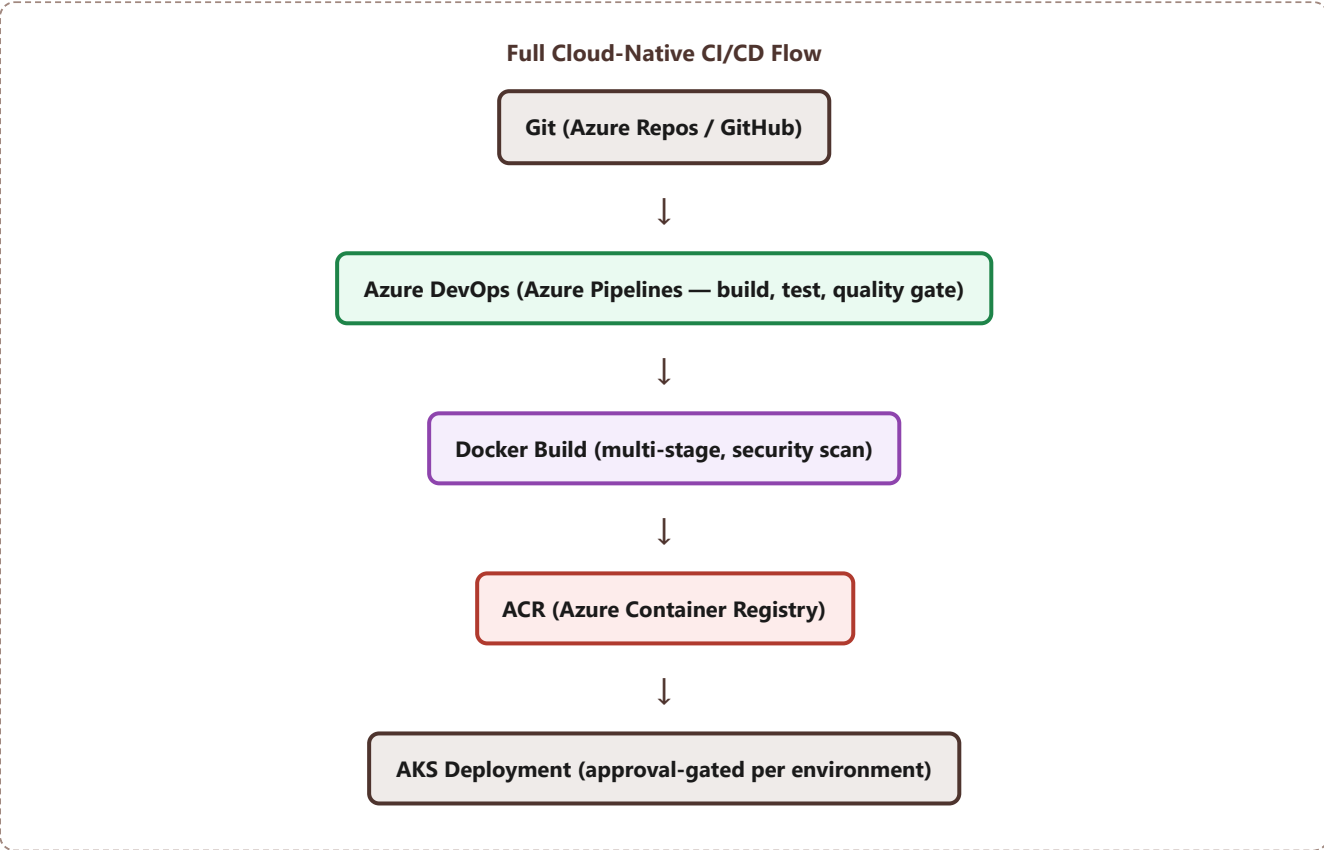
Tool	Purpose in This Architecture
Azure Monitor	The umbrella platform — metrics, alerts, and dashboards across every Azure resource in the architecture
Application Insights	Distributed tracing and APM — auto-instruments every Spring Boot service's requests, dependencies, exceptions
Grafana	Custom dashboards, often layered on top of Azure Monitor Managed Prometheus for Kubernetes-specific metrics
Splunk	Centralized log search across all services, often still used even in full-cloud setups if the organization already standardized on Splunk (Observability guide, Section 13)

17.2 Why Full Cloud Doesn't Eliminate the Need for These Tools — It Just Changes Who Maintains Them

The exact same observability need exists in every model (Traditional, Hybrid, Full Cloud) — WHAT changes is that Azure Monitor/Application Insights are fully managed services requiring zero patching/scaling effort from your team, whereas the same capability required dedicated self-hosted infrastructure in the Traditional model (Section 7).

18. Full Cloud CI/CD Flow (Git → Azure DevOps → Docker Build → ACR → AKS Deployment)

18.1 The Complete Real Pipeline



This is EXACTLY the same real pipeline already covered in complete depth with real YAML in the CI/CD guide (Section 27.2) — this section exists purely to place it correctly within this guide's overall architecture comparison, confirming CI/CD is a first-class, integral part of the Full Cloud-Native model rather than an afterthought.

18.2 What's Genuinely Different From the On-Premises Jenkins Pipeline (Section 5)

Aspect	On-Premises (Jenkins)	Full Cloud (Azure DevOps)
CI server maintenance	Your team patches/scales Jenkins itself	Azure Pipelines is fully managed — Microsoft maintains it
Build agent capacity	Fixed number of Jenkins agents, manually scaled	Microsoft-hosted agents scale elastically on demand
Deployment target	A fixed set of physical/virtual servers	AKS, which itself auto-scales based on real demand
Approval gates	Often manual runbook sign-off	Native Environment-based Approvals and Checks (CI/CD guide, Section 17.6)

19. Traditional vs Hybrid vs Full Cloud — Full Comparison (Cost, Performance, Maintenance)

19.1 Cost Comparison

Aspect	Traditional On-Premises	Hybrid	Full Cloud-Native
Upfront capital cost	Very high — buy all hardware upfront	Medium — some new cloud spend, existing hardware retained	Low — no hardware purchase, pay-as-you-go
Ongoing operational cost	Fixed (staff, power, cooling) regardless of actual usage	Mixed — on-prem fixed costs + variable cloud costs	Fully variable — scales precisely with actual usage
Cost predictability	Very predictable (fixed budget)	Moderately predictable	Can be less predictable without active cost governance/monitoring
Idle capacity waste	High — peak-provisioned hardware sits idle most of the time	Reduced for cloud-hosted portions	Minimal — auto-scaling matches capacity to real demand

19.2 Performance Comparison

Aspect	Traditional On-Premises	Hybrid	Full Cloud-Native
Global user latency	Poor for geographically distant users (no CDN/edge presence)	Improved for cloud-hosted frontend/API layers	Excellent — global CDN/edge network (Front Door/CloudFront)
Scaling responsiveness	Slow — physical hardware procurement required	Fast for cloud portions, slow for on-prem portions	Fast — auto-scaling reacts within minutes or seconds
Cross-environment latency	N/A — everything is local	Real added latency crossing the hybrid boundary (Section 10.2)	N/A — everything within one cloud provider's backbone network

19.3 Maintenance Comparison

Aspect	Traditional On-Premises	Hybrid	Full Cloud-Native
Physical hardware maintenance	Fully your responsibility	Partial — only for the on-prem portion	None — cloud provider's responsibility entirely
OS/patching burden	Full — every server patched by your team	Partial — only on-prem servers	Greatly reduced — managed services (PaaS/serverless) require minimal to no OS patching
Required team expertise breadth	Very broad — networking, hardware, OS, storage, plus application skills	Broad — both on-prem AND cloud skills needed	Narrower physical/infra skills needed; deeper cloud-service-specific expertise needed instead

Disaster recovery complexity	High — requires building/maintaining a second physical site	Moderate — cloud can serve as the DR target for on-prem	Low — multi-region deployment is largely a configuration exercise
------------------------------	---	---	---

19.4 The Honest Trade-Off Summary

Interview one-liner: "There's no universally 'best' model — Traditional On-Premises offers maximum control and predictable fixed costs but high operational burden and poor global reach; Full Cloud-Native offers elasticity, global reach, and minimal infrastructure burden but requires strong cloud governance to avoid cost surprises and deep cloud-specific expertise; Hybrid is a deliberate middle ground, often driven by regulatory constraints or as a genuine, real transitional migration phase."

20. Migrating On-Premises to Azure — Migration Strategies (The 6 R's)

20.1 The Six Real Migration Strategies

Strategy	What It Means	Real Example
Rehost ("Lift and Shift")	Move the application AS-IS onto cloud VMs, minimal changes	Moving an existing Spring Boot WAR from an on-prem Tomcat server directly onto an Azure VM running Tomcat
Replatform ("Lift, Tinker, and Shift")	Make small optimizations during the move without changing core architecture	Moving the same app to Azure App Service instead of a raw VM — same app, less infrastructure to manage
Repurchase	Replace with a SaaS/managed alternative	Replacing a self-hosted email server with a managed email service
Refactor / Re-architect	Redesign the application to be truly cloud-native (microservices, containers, managed databases)	Breaking a monolith into Spring Boot microservices deployed on AKS, exactly per Section 13's target architecture
Retire	Decommission applications no longer needed	An old reporting tool superseded by a modern Power BI/Grafana dashboard
Retain	Deliberately keep some systems on-premises (Hybrid, Section 8)	Core ledger database kept on-prem due to regulatory data-residency requirements

20.2 Real Strategy Selection Per Component

Real Migration Strategy Mapping for Our Banking Example

React Frontend → Refactor (to Azure Static Web App + CDN)

Spring Boot Monolith → Refactor (break into microservices, containerize, deploy to AKS)

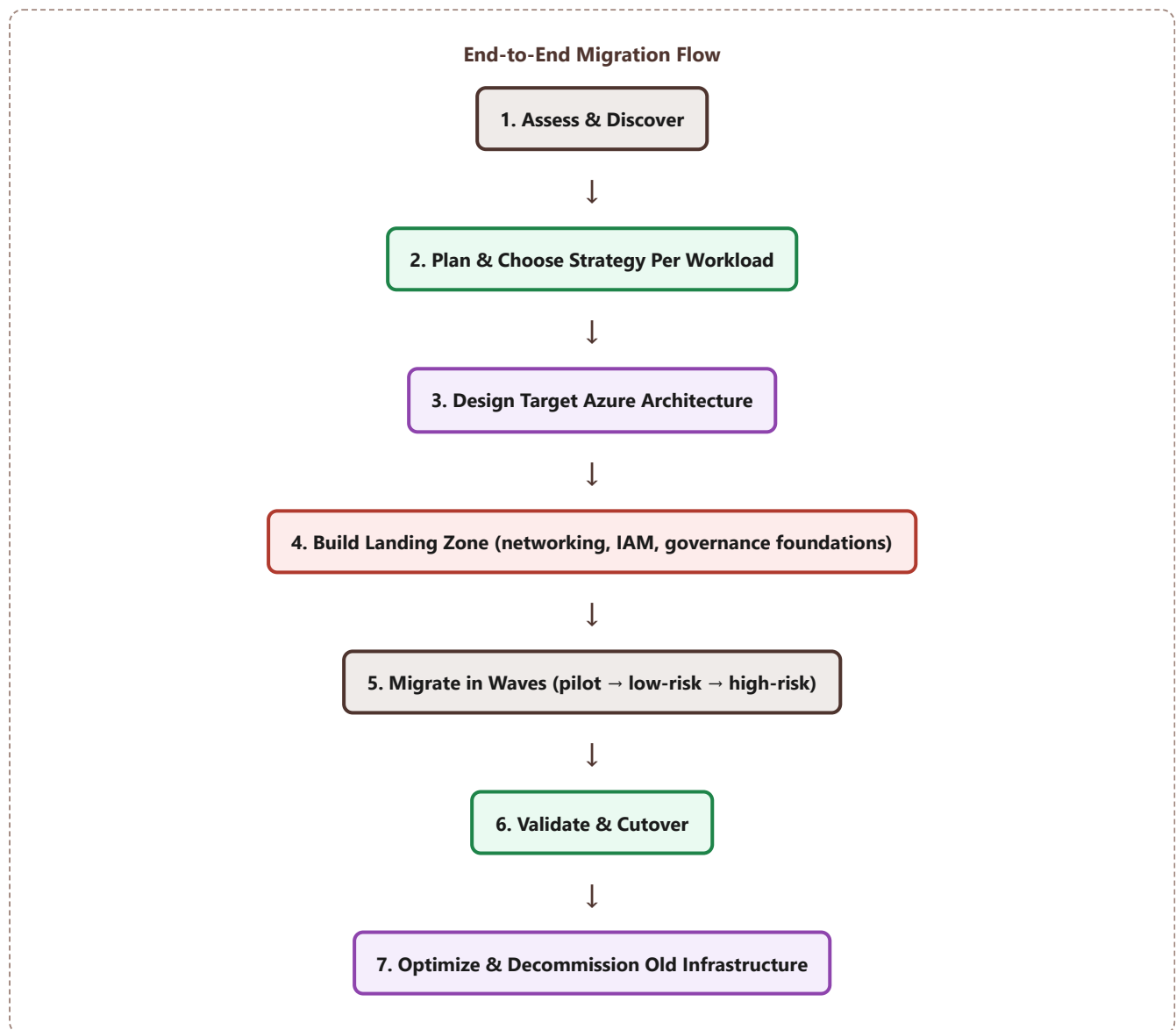
Legacy Reporting Tool → Retire (replace with Grafana/Power BI)

Core Ledger Database → Retain (on-prem, for now — regulatory) or Replatform (managed DB, if regulation allows)

Real scenario: "Our organization used FOUR different strategies simultaneously across one migration program — Rehost for a low-priority legacy internal tool (fast, cheap, low risk), Refactor for our core customer-facing Spring Boot services (worth the investment for long-term agility), Retire for two reporting tools nobody used anymore, and Retain for the core ledger database (regulatory constraint) — real migrations are rarely one single strategy applied uniformly."

21. Migration Steps End to End

21.1 The Complete Real Migration Journey



21.2 Step 1 — Assess & Discover

Inventory every application, its dependencies, data flows, and current resource utilization — real organizations use tools like Azure Migrate to automatically discover on-premises VMs/servers and assess their cloud readiness and estimated cost.

21.3 Step 2 — Plan & Choose Strategy Per Workload

Apply the 6 R's (Section 20) to EACH individual application/service — not a single blanket decision for the whole organization.

21.4 Step 3 — Design Target Azure Architecture

Design the target state (Section 13's full architecture diagram) — which services need AKS vs App Service, which databases map to Azure SQL vs Cosmos DB, and how the CI/CD pipeline will work going forward (Section 18).

21.5 Step 4 — Build Landing Zone

Establish the foundational Azure infrastructure BEFORE migrating any workload — VNets/subnets (AWS/Azure guide, Section 8), Entra ID setup, RBAC policies, Key Vault, and governance/cost-management guardrails — a stable, secure foundation everything else builds on top of.

21.6 Step 5 — Migrate in Waves

Wave	What Migrates	Why This Order
Pilot Wave	One small, low-risk, non-critical application	Validates the process, tooling, and team's cloud skills with minimal business risk
Low-Risk Wave	Stateless services, internal tools	Builds confidence and momentum with genuinely low-stakes workloads
High-Risk/Core Wave	Core customer-facing services, critical databases	Tackled last, once the team has real, proven migration experience from earlier waves

21.7 Step 6 — Validate & Cutover

Run the new cloud environment in PARALLEL with the old on-premises system where possible (exactly the Strangler Fig pattern, Microservices guide Section 9), gradually shifting real traffic over (often via DNS weighting or a router splitting traffic by percentage) rather than an instant, risky "big bang" cutover.

21.8 Step 7 — Optimize & Decommission

Once the cloud environment is proven stable under real production load, decommission the old on-premises infrastructure (or repurpose it), and begin ongoing cost/performance optimization — cloud migration isn't a one-time event, it's followed by continuous tuning (right-sizing VMs, adjusting auto-scaling thresholds, reviewing storage tiers).

22. Migration Real Scenarios, Issues & Solutions

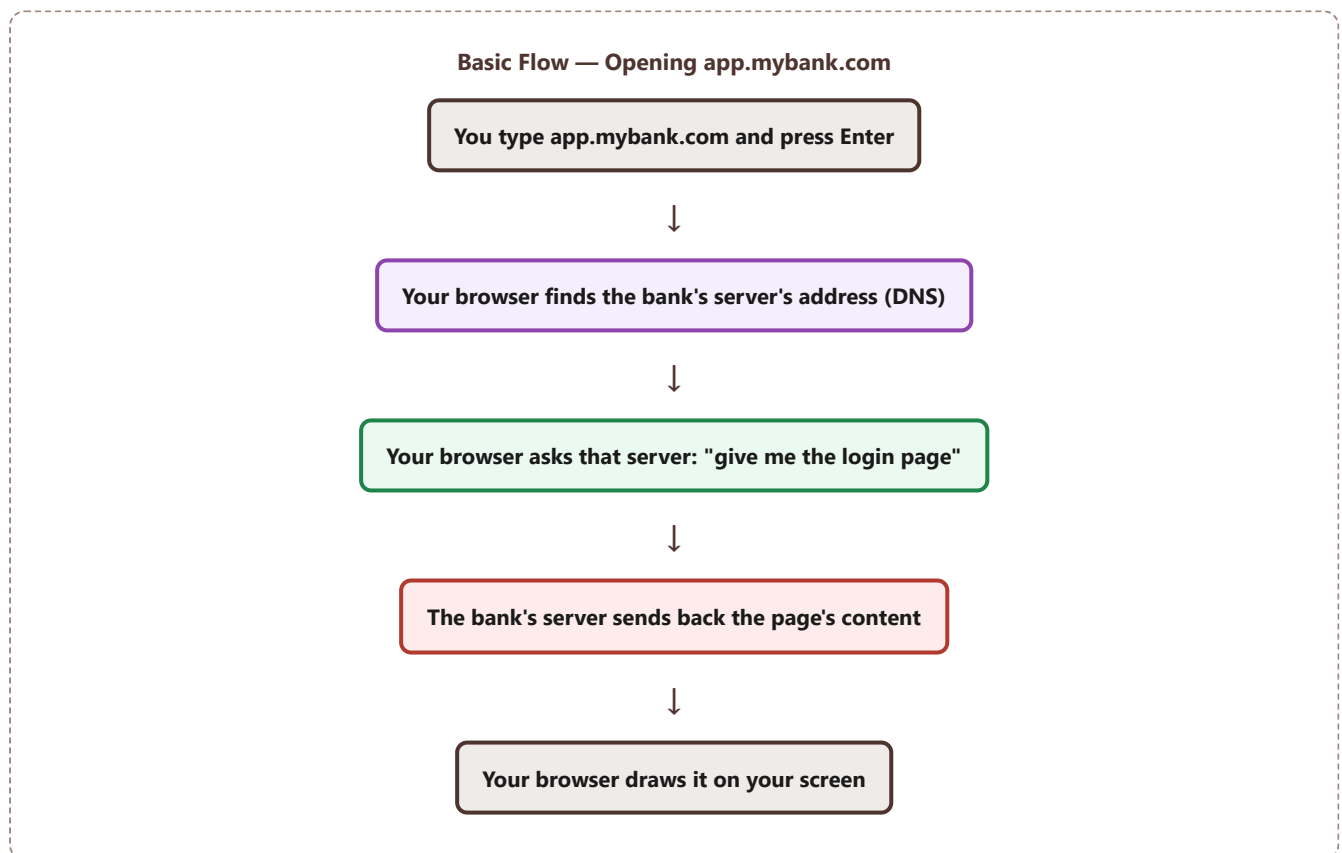
Issue	Cause	Real Solution
Migrated application performs worse in the cloud than on-prem	"Lift and shift" without any re-architecture — cloud VM sizing/network topology mismatched to the app's actual needs	Right-size VM/AKS resources based on real observed metrics; consider a Replatform/Refactor pass after initial Rehost
Unexpectedly high cloud costs post-migration	No cost governance/tagging, resources left running without auto-scaling or lifecycle policies	Implement cost management tooling, enforce auto-scaling and storage tiering (AWS/Azure guide, Sections 10.6, 12)
Data synchronization issues during a phased cutover	Both old (on-prem) and new (cloud) systems accepting writes simultaneously without proper sync	Use Change Data Capture (CDC) or a clear single-writer cutover boundary during the transition window
Team struggles with unfamiliar cloud-native tooling	Insufficient training/ramp-up time before migration began	Invest in training and start with a genuinely low-risk pilot wave (Section 21.6) to build real hands-on experience first
A "big bang" cutover caused a major outage	No parallel-running validation period, all traffic switched instantly	Use a phased/percentage-based traffic cutover with an immediate rollback plan, per the Strangler Fig pattern
Compliance/audit concerns post-migration	Data residency or access control requirements not fully mapped to the new cloud environment's configuration	Involve compliance/security teams from the Assess phase (Section 21.2) onward, not as an afterthought

23. What Happens When You Open a URL — Basic Introduction With a Real Example

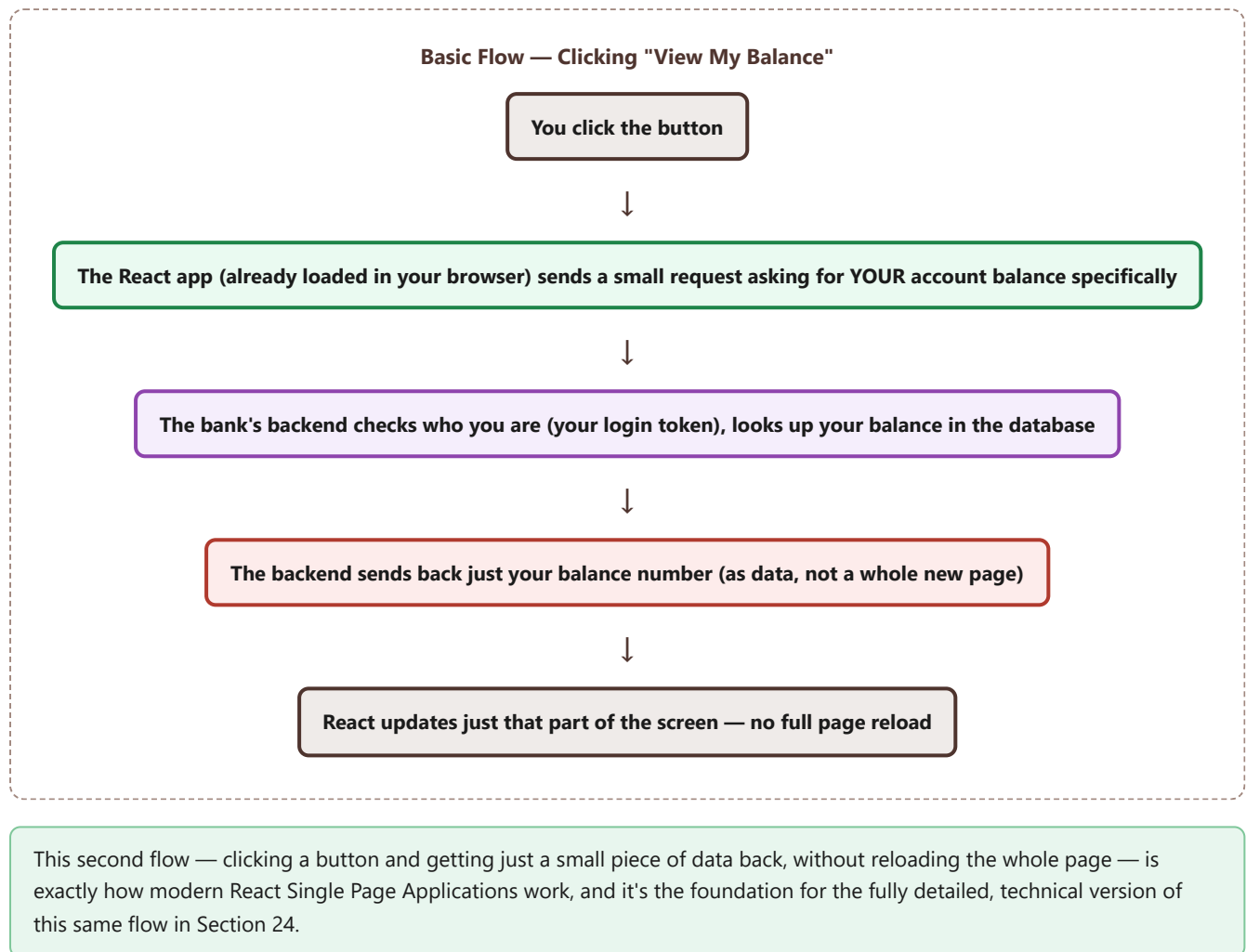
23.1 The Simple Version, In Plain Language

Let's use a real example: you open your banking app and type `app.mybank.com`. In the simplest possible terms — your browser finds the bank's server on the internet, asks it for the login page, the server sends it back, and your browser displays it. Everything else in this guide is really just filling in the details of exactly HOW that happens.

23.2 The Real Basic Flow



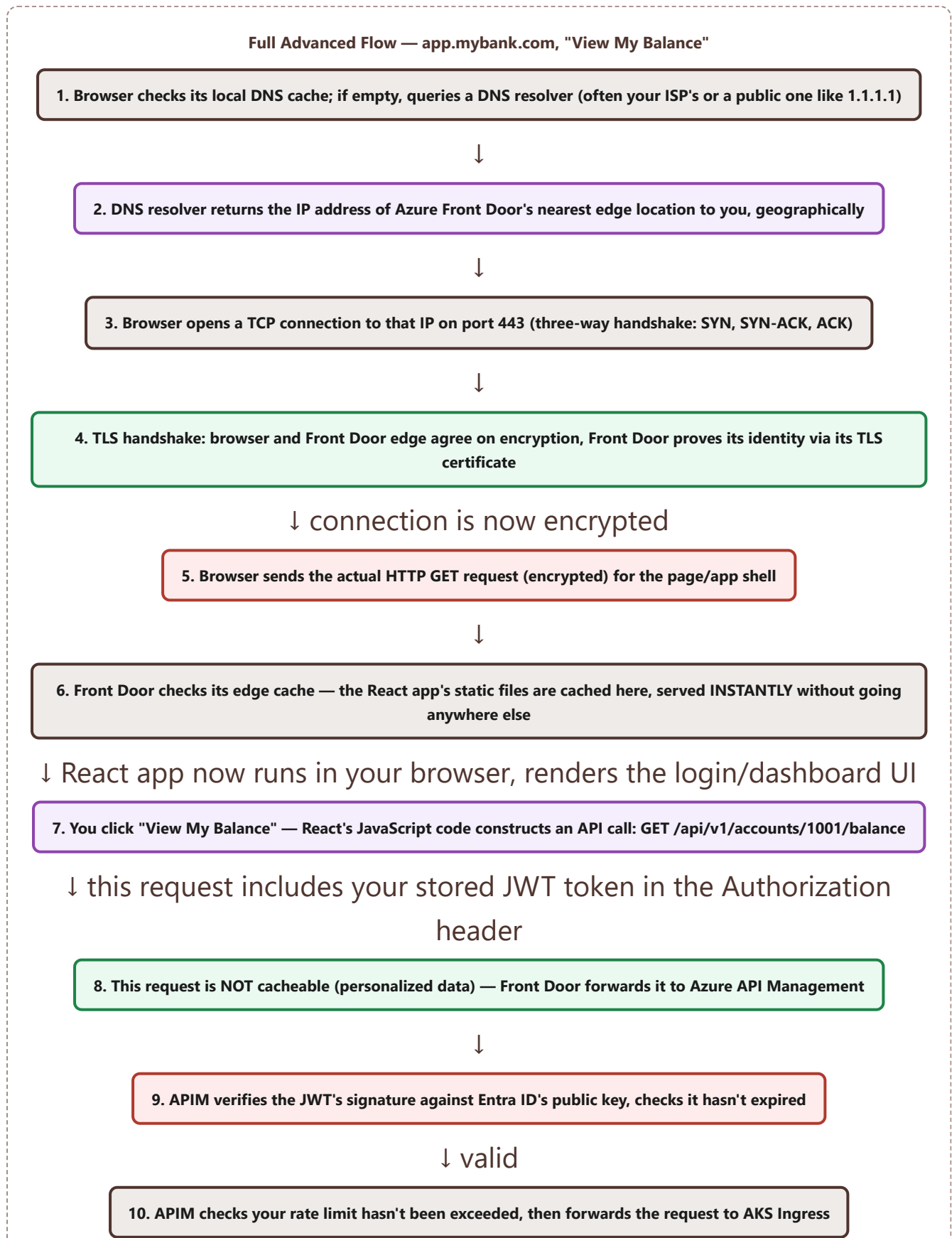
23.3 What Happens When You Then Click "View My Balance"

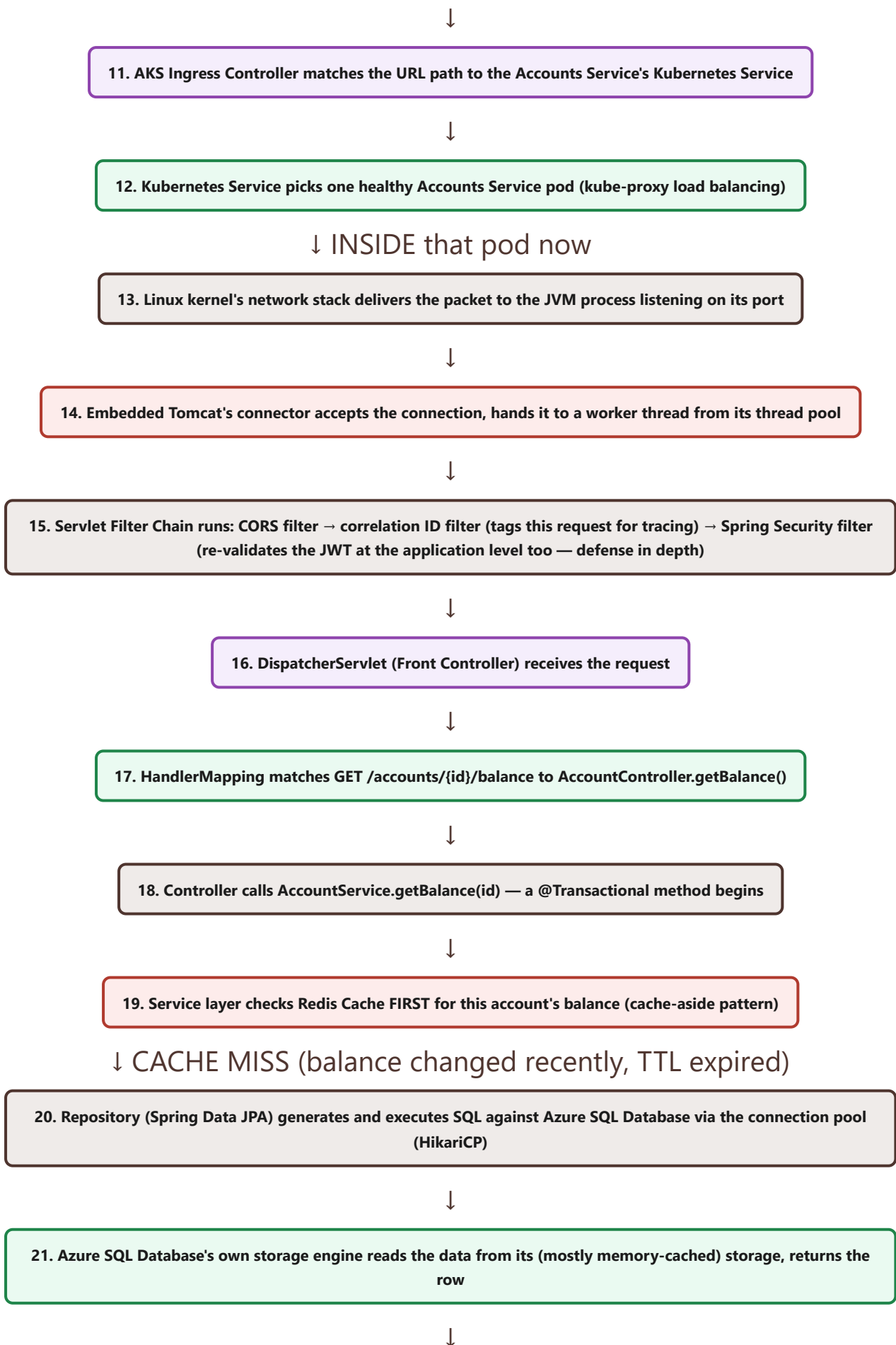


24. What Happens When You Open a URL — Full Advanced

Internal Working

24.1 The Complete Technical Journey, Every Layer





22. Hibernate maps the ResultSet to an Account entity object; Service maps it to an AccountDto; the result is written into Redis Cache for next time

↓ transaction commits

23. Controller returns the DTO; Jackson serializes it to JSON

↓

24. Response flows back OUT through the same filter chain, then the pod, Ingress, APIM, Front Door, and finally to your browser

↓

25. React receives the JSON, updates its component state, re-renders JUST the balance display on your screen

↓ throughout ALL 24 steps above

26. Application Insights recorded a distributed trace span at every single hop, all tagged with one shared trace ID, viewable as a single connected timeline

24.2 Total Real-World Elapsed Time

Despite this being 26 distinct conceptual steps across many different systems, the ENTIRE flow — from your click to seeing your updated balance — typically completes in well under 200 milliseconds in a healthy, well-architected system, often under 100ms on a cache hit. Understanding this full depth is exactly what separates "I can write a Spring Boot controller" from "I understand how my code actually reaches a real user, end to end, in production."

25. Interview Questions & Real-Time Scenario-Based Q&A

Q1. What are the three main deployment models covered in this guide, and how do they fundamentally differ?

Traditional On-Premises (everything self-hosted in your own data center), Hybrid (some workloads on-prem, some in the cloud, connected via VPN/ExpressRoute), and Full Cloud-Native (everything hosted and managed by a cloud provider like Azure). They differ primarily in who owns/manages infrastructure, cost model (fixed capital vs pay-as-you-go), and scaling flexibility.

Q2. Why might a large regulated organization deliberately choose to remain Hybrid rather than fully migrating to the cloud?

Regulatory/data-residency requirements mandating certain data remain within a specific physical jurisdiction or under direct organizational control, existing capital investment in on-prem hardware, or genuine legacy system migration risk/cost — Hybrid can be either a permanent deliberate architecture or a temporary transitional phase.

Q3. What's the real difference between a Site-to-Site VPN and Azure ExpressRoute for hybrid connectivity?

A Site-to-Site VPN is an encrypted tunnel over the public internet — cheaper but subject to variable latency/congestion. ExpressRoute is a dedicated, private physical circuit directly into Azure, never touching the public internet — more expensive, but far more predictable latency and stronger security/compliance posture.

Q4. Explain the 6 R's of cloud migration strategy with a real example of when you'd use each.

Rehost (lift-and-shift a low-priority app as-is), Replatform (move to a managed service like App Service with minor changes), Repurchase (replace with a SaaS alternative), Refactor (re-architect into cloud-native microservices for long-term agility), Retire (decommission unused applications), and Retain (deliberately keep something on-premises, e.g., due to regulation).

Real scenario: applying Rehost to a low-priority internal tool, Refactor to core customer-facing services, Retire to unused reporting tools, and Retain for a regulated core database — all within one migration program.

Q5. Why do real migrations use a phased "wave" approach instead of migrating everything at once?

A pilot wave with a low-risk application validates the process and builds team confidence/skills with minimal business risk; low-risk workloads follow next; core/critical systems are migrated last, once the team has real, proven experience — avoiding the high risk of a single "big bang" migration of everything simultaneously.

Q6. What real cost differences exist between Traditional On-Premises and Full Cloud-Native models?

On-premises requires very high upfront capital cost (buying hardware for peak capacity that mostly sits idle) with predictable fixed ongoing costs regardless of actual usage. Full Cloud-Native has minimal upfront cost and fully variable, usage-based pricing, which can be cheaper overall but requires active cost governance to avoid unexpected spend from unused/over-provisioned resources.

Q7. Walk through what happens, in full technical detail, when a user clicks a button in a React app that calls a backend API.

React's JavaScript constructs an API call including the stored JWT token; if not cacheable, it's forwarded through the CDN/Front Door to the API Gateway/APIM, which validates the JWT and applies rate limiting, then routes through the Kubernetes Ingress to a load-balanced Service, into a specific Spring Boot pod, through its filter chain (security, correlation ID), DispatcherServlet, Controller, a @Transactional Service layer (checking cache first), Repository/JPA hitting the database on a cache miss, and the response flows back out through the exact same path, all captured end to end by a single distributed trace ID.

Q8. Why is defense-in-depth security important across this full request flow, rather than relying on just one layer?

Each layer (WAF at the edge, JWT validation at the API Gateway, network policies within Kubernetes, Spring Security within the application, database-level access control) provides independent protection — if any single layer is misconfigured or bypassed, the others still provide protection, rather than the entire system's security depending on one single point of failure.

Q9. What's the real difference in observability/monitoring maintenance burden between On-Premises and Full Cloud models?

On-premises requires your own team to install, patch, scale, and ensure high availability of the observability stack itself (Splunk, Grafana, Zipkin servers) — the monitoring tools themselves need monitoring. In Full Cloud, Azure Monitor/Application Insights are fully managed services requiring no infrastructure maintenance from your team at all.

Q10. Describe a real scenario where a migration project ran into unexpected performance problems, and how you'd diagnose/fix it.

A "lift and shift" migration performed worse in the cloud than on-premises because the VM sizing and network topology weren't adjusted for the cloud environment's actual characteristics. Diagnosed via real observed CPU/memory/network metrics post-migration, then fixed by right-sizing resources and considering a Replatform pass (e.g., moving to a managed PaaS service) rather than assuming a pure like-for-like VM migration would automatically perform equivalently.

Q11. Why would an organization keep its core database on-premises while moving its frontend and API layer to the cloud (as in the Hybrid example)?

This "Frontend-Cloud, Data-On-Prem" pattern delivers immediate, lower-risk user experience improvements (global CDN, elastic API scaling) without needing to solve the much harder, higher-risk problem of migrating a critical, deeply embedded legacy database — the database migration can proceed carefully and separately, on its own timeline.

Q12. What real latency consideration must a service account for when calling across a Hybrid boundary (cloud to on-premises)?

Every call crossing the ExpressRoute/VPN boundary pays a real, measurable round-trip cost higher than intra-cloud or intra-datacenter calls — real teams mitigate this by caching frequently-accessed, rarely-changing reference data locally (e.g., in Redis within the cloud environment) to avoid unnecessarily crossing that boundary on every single request.

26. Final Summary & Cheat Sheet

Traditional On-Premises: full control, predictable fixed costs, but high operational burden, slow scaling, poor global reach, and every layer (network, hardware, OS, observability tooling) is your own team's responsibility.

Hybrid: a deliberate or transitional split between on-prem and cloud, connected via VPN/ExpressRoute, driven by regulatory constraints, legacy dependencies, or as a genuine phased migration path — with real added latency and complexity at the boundary that must be actively managed.

Full Cloud-Native: React on Azure Static Web Apps, Spring Boot microservices on AKS, databases as managed Azure SQL/Cosmos DB/Redis services, all secured via Key Vault/Entra ID, observed via Azure Monitor/Application Insights, and delivered continuously via Azure DevOps/ACR/AKS — elastic, globally reachable, with infrastructure maintenance burden largely shifted to the cloud provider.

The full request lifecycle: from DNS resolution and TLS handshake, through CDN edge caching, API Gateway authentication/rate-limiting, Kubernetes routing, the complete internal Spring Boot request pipeline (filters → DispatcherServlet → Controller → Service → cache-aside → Repository/database), all the way back to the user's screen — typically completing in well under 200ms, entirely traced end to end by a single correlation ID.

Migration: apply the 6 R's per-workload (not uniformly), build a solid landing zone first, migrate in waves starting with low-risk pilots, validate via parallel running before cutover, and treat optimization as continuous, not a one-time event.

26.1 Quick Cheat Sheet

Term	One-Line Meaning
On-Premises	Everything self-hosted in your own data center
Hybrid	Some workloads on-prem, some in the cloud, connected together
Full Cloud-Native	Everything hosted/managed by a cloud provider
ExpressRoute	Dedicated private circuit connecting on-prem to Azure, bypassing the public internet
Rehost	Lift-and-shift migration with minimal changes
Refactor	Re-architect into a truly cloud-native design during migration
Retain	Deliberately keep a workload on-premises
Migration Wave	A phased group of applications migrated together, ordered by risk
Landing Zone	The foundational cloud infrastructure/governance setup built before migrating workloads
Defense in Depth	Multiple independent security layers, so no single failure exposes the whole system
Cache-Aside	Check the cache first, populate it only on a miss

Use this guide alongside the AWS ↔ Azure Cloud Architecture guide, the Docker & Kubernetes guide, the Microservices guide, the CI/CD guide, the Observability & Monitoring guide, and the Spring & Spring Boot guide — together they cover the complete real-world Java full-stack developer journey from understanding deployment models and infrastructure evolution, through the exact internal mechanics of a single request, to planning and executing a real cloud migration.